UNIVERSIDADE DE SANTA CRUZ DO SUL

Curso de Ciência da Computação

DOCUMENTO DE ARQUITETURA DE SOFTWARE

PSA Gestor

Sistema de Gestão da Aplicação de Recursos do Fundo Municipal
de Preservação Ambiental

ENTREGA 3 - Arquitetura + Diagramas UML

Versão 2.0 - 09/10/2026

Disciplina: Análise de Projeto de Sistemas
Projeto Integrador - Grupo 2
Professor: Kurt Werner Molz

Equipe:

Gabriel Dall'Acqua Gomes
João Vitor Meinhardt Swarowsky
Josué Santos Schmidt
Mateus Haetinger
Rafael Lamonatto Jackisch

Santa Cruz do Sul - RS
2026

Identificação do Documento

Campo	Identificação
Projeto	PSA Gestor - Gestão da Aplicação de Recursos do Fundo Municipal de Preservação Ambiental
Organização	Centro Socioambiental - Universidade de Santa Cruz do Sul (UNISC)
Data / versão	09/10/2026 / 2.0 - Entrega 3
Finalidade	Apresentar a estrutura do sistema, suas interfaces, implantação, organização do código e decisões arquiteturais.

Histórico de Revisões

Versão	Data	Principais alterações
1.0	07/10/2026	Modelagem inicial da arquitetura, dos dados e dos fluxos do PSA Gestor.
2.0	09/10/2026	Consolidação da arquitetura em VPS, interfaces, pacotes, sequências UML e decisões arquiteturais. Definição de cache seletivo, segurança, desempenho e recuperação.

Sumário

1. Visão geral e padrão arquitetural 3
2. Diagrama de componentes 4
3. Componentes e contratos de interface 5
4. Diagrama de implantação 6
5. Diagrama de pacotes 7
6. ADR-001 - Padrão arquitetural 8
7. ADR-002 e ADR-003 - Dados e arquivos 9
8. Atributos de qualidade 10
9. Rastreabilidade entre requisitos e arquitetura 11
10. ADR-004 - Cache seletivo com Redis 12
11. Recursos e desempenho 13
12. Segurança e publicação web 14
13. Operação e recuperação 15
14. Sequência de consulta com cache 16
15. Sequência de pagamento 17
16. Decisões consolidadas 18

1. Visão geral e padrão arquitetural

O PSA Gestor centraliza o cadastro de produtores e propriedades, o diagnóstico ambiental, o cálculo do incentivo, a geração de contratos e a gestão financeira simulada do Programa de Pagamento por Serviços Ambientais. Este documento apresenta a estrutura dos componentes e suas interfaces, a distribuição dos artefatos no ambiente de execução, a organização dos pacotes e as decisões arquiteturais que sustentam os requisitos funcionais e de qualidade.

1.1 Padrão arquitetural

A arquitetura adota um **monólito modular em camadas**, com frontend Angular 21 separado e uma única aplicação backend Spring Boot. O backend organiza as funcionalidades por domínio e separa, em cada módulo, a entrada HTTP, as regras de negócio e a persistência. Os módulos comunicam-se por chamadas locais; somente o frontend e os serviços externos utilizam interfaces de rede.

A separação em camadas concentra validações e transações no backend, enquanto a organização por funcionalidade mantém próximos os elementos de cada domínio. Essa estrutura reduz a complexidade operacional e permite controlar aprovação, reserva de recursos e pagamento por transações no MariaDB. Em contrapartida, a publicação e a escala do backend ocorrem em conjunto. O ADR-001 detalha a justificativa e as alternativas.

Camada	Responsabilidade
Apresentação	Angular: formulários e navegação. Filtros de segurança: autenticação. Controllers: DTOs, validação de entrada e respostas HTTP.
Negócio	Serviços de aplicação e modelo de domínio: diagnóstico, fórmula, contratos, aportes e regras financeiras. Controlam as transações.
Dados e integrações	JPA: persistência relacional. Redis: cópias temporárias de indicadores. R2: arquivos privados. Os adaptadores isolam os detalhes de acesso a esses serviços.

Quadro 1 - Responsabilidades das camadas.

1.2 Perspectivas da arquitetura

Visão do modelo 4+1	Representação nesta versão
Desenvolvimento	Componentes, interfaces e pacotes - seções 2, 3 e 5.
Física	Nós de execução, artefatos e comunicação - seção 4.
Lógica	Responsabilidades dos módulos e regras de domínio - seções 3 e 8.
Processo	Transações e interações em execução - seções 7, 14 e 15.
+1 Cenários	Casos de uso e histórias da especificação 2.1, relacionados aos módulos na seção 9.

O escopo mantém croqui simplificado, assinatura simulada e aportes manuais. Não contempla integração bancária, integração automática com o Fundo Municipal ou MUDA, SIG completo nem sincronização offline.

2. Diagrama de componentes

A fronteira delimita a aplicação. As interfaces separam responsabilidades de negócio dos mecanismos de autenticação, persistência, armazenamento e cache. As dependências de infraestrutura ficam fora dessa fronteira.

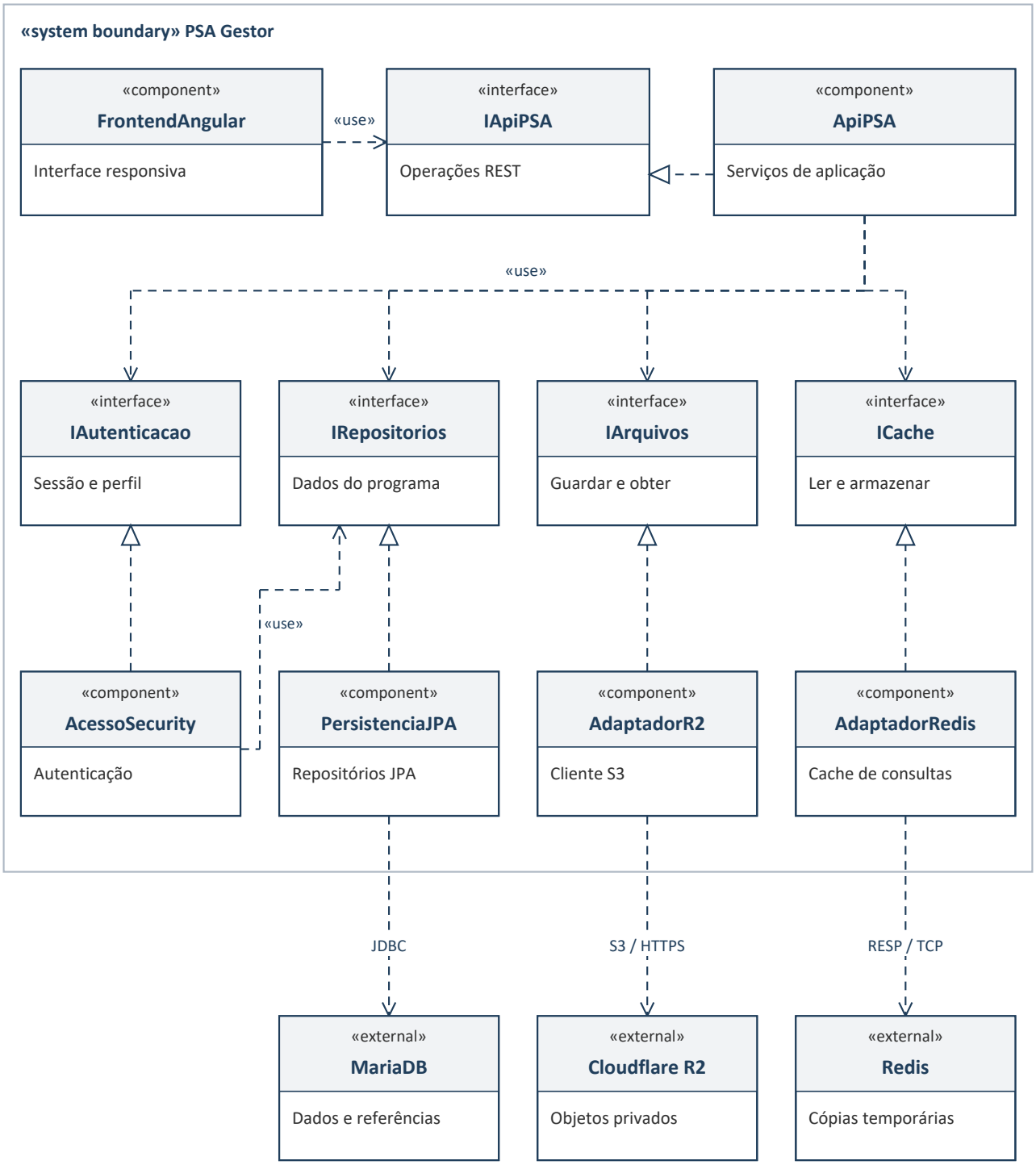


Figura 1 - Componentes, interfaces fornecidas e dependências de uso.

Setas tracejadas abertas indicam uso; triângulos vazios indicam realização da interface pelo provedor. IRepositorios representa contratos por entidade. Apenas consultas elegíveis usam ICache. Os componentes do backend pertencem ao mesmo artefato Java.

3. Componentes e contratos de interface

Componente	Responsabilidade / interfaces
FrontendAngular	Apresenta telas por perfil; requer IApiPSA. Não acessa diretamente o banco ou o bucket R2.
ApiPSA	Fornece IApiPSA; coordena os casos de uso nos módulos abaixo e requer autenticação, persistência, arquivos e cache seletivo.
AcessoSecurity	Fornece IAutenticacao: autenticar, consultar sessão/perfil e encerrar sessão. Requer persistência de usuários.
PersistenciaJPA	Fornece IRepositorios e utiliza MariaDB via JDBC. A interface representa uma família de contratos de repositório por entidade, não um repositório genérico único.
AdaptadorR2	Fornece IArquivos: guardar, obter e remover objetos em compensação; utiliza a API compatível com S3 do R2.
AdaptadorRedis	Fornece ICache para indicadores não financeiros. Expiração e falha do cache não alteram a persistência de negócio.

Quadro 2 - Responsabilidades dos componentes da Figura 1.

3.1 Módulos de negócio da ApiPSA

Módulo	Responsabilidade principal
cadastros	Produtores, propriedades, fontes hídricas e visitas; registrar ausência de fontes explicitamente.
configuracao	Parâmetros e pesos por projeto, sob administração; validar configuração completa.
diagnostico	Respostas progressivas, conclusão, correção e apuração automática da pontuação.
calculo	Aplicar fórmula e preservar os parâmetros efetivamente utilizados.
contratos	Preparar documento, vincular croqui/cálculo e controlar aprovação, assinatura simulada e vigência.
financeiro	Registrar aportes, compromissos e pagamentos parciais; consolidar saldos.
consultas	Relatórios, indicadores e histórico restrito do produtor.

3.2 Contratos e direção das dependências

IApiPSA é a interface HTTP sob /api/v1: JSON para dados, multipart para croquis e conteúdo binário nos downloads. IAutenticacao, IRepositorios, IArquivos e ICache representam contratos internos do backend. Requisições inválidas não seguem para gravação; entidades JPA e credenciais não integram os DTOs de saída.

Os módulos de negócio utilizam os contratos de dados e arquivos. Os adaptadores não chamam controllers nem dependem do Angular. Essa direção permite substituir o armazenamento de arquivos sem alterar os casos de uso e evita dependências circulares. A organização detalhada aparece na Figura 3.

4. Diagrama de implantação

A implantação prevista concentra os serviços da aplicação em uma VPS da Loophost, empresa de Taquara/RS, com IP público fixo. O R2 permanece externo. Os nomes DuckDNS deverão apontar para esse IP; a resolução DNS não encaminha tráfego HTTP. O driver JDBC acessará o MariaDB pelo protocolo do banco sobre TCP.

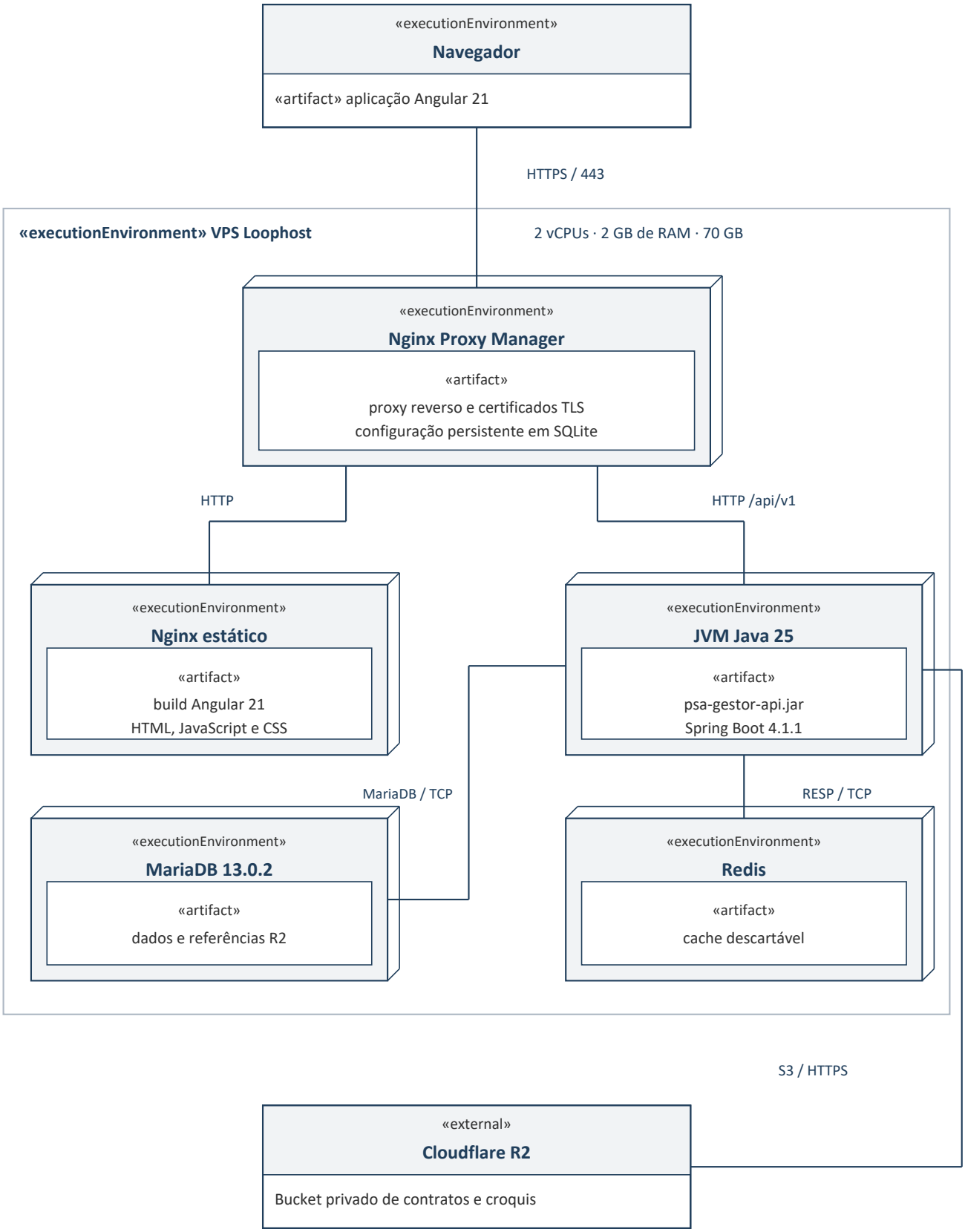


Figura 2 - Nós de execução, artefatos e canais de comunicação.

5. Diagrama de pacotes

O namespace adotado é br.unisc.psagestor, organizado por funcionalidade. As setas indicam dependências dos serviços de origem em contratos do pacote de destino. A figura destaca os caminhos principais; acessos de consulta e dependências transversais não estão todos representados.

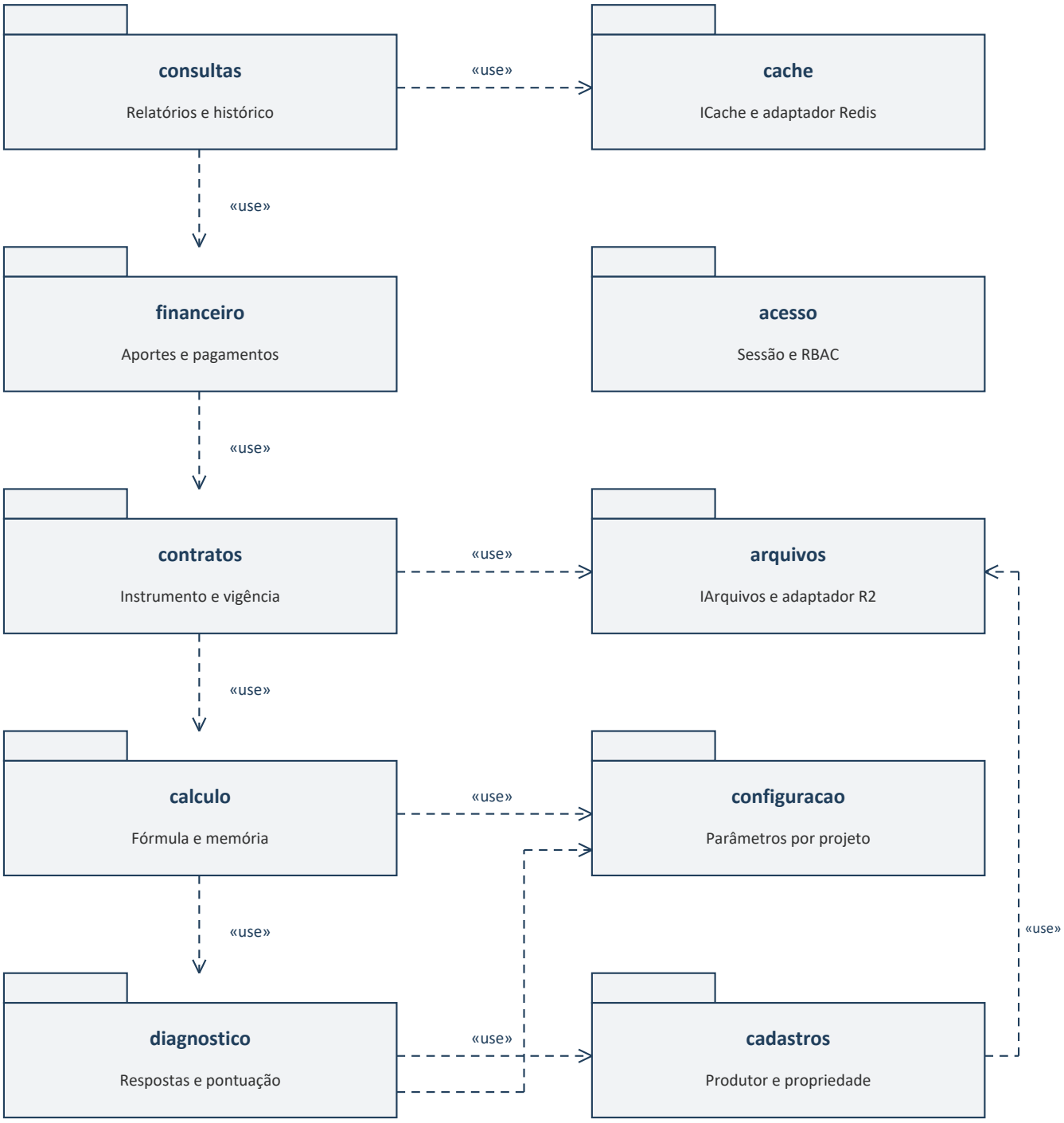


Figura 3 - Diagrama UML de pacotes. Dependências principais sem ciclos.

Cada funcionalidade separa api (controllers e DTOs), service, repository e model. api depende de service; service utiliza repository e model; repository utiliza model. A segurança atua na entrada das requisições. O pacote comum reúne tipos compartilhados e foi omitido, assim como dependências repetidas de persistência. O pacote cache não depende de consultas.

Financeiro coordena aprovação e reserva, consultando contratos sem dependência inversa. Contratos utiliza cálculo e arquivos; diagnóstico utiliza cadastro e configuração. Consultas acessa ICache para indicadores; cadastros acessa IArquivos para croquis.

6. ADR-001 - Padrão arquitetural

Monólito modular em camadas

Status: Adotado nesta arquitetura · **Registro:** 09/10/2026

Contexto

O ciclo do PSA exige integração entre cadastro, diagnóstico, cálculo, contrato e movimentações financeiras. A aprovação e a reserva de recursos precisam manter consistência, assim como os pagamentos parciais. O ambiente de execução definido possui uma VPS com 2 vCPUs, 2 GB de RAM e 70 GB de armazenamento; o escopo não estabelece disponibilidade formal nem necessidade de escalabilidade independente por módulo.

Alternativas consideradas

Alternativa	Avaliação
Monólito modular em camadas	Separa responsabilidades e mantém transações no mesmo banco; implantação única e comunicação local entre módulos.
Microsserviços	Permitem publicação e escala independentes, mas exigem múltiplos processos, comunicação remota e tratamento de consistência distribuída para o ciclo financeiro.
Eventos distribuídos com mensageria	A alternativa assíncrona considerada exige infraestrutura de mensagens e tratamento de reentrega e consistência eventual. Eventos locais síncronos não exigem essa infraestrutura.

Decisão

Adota-se uma aplicação backend Spring Boot organizada em módulos de negócio e camadas, com frontend Angular 21 separado. As operações entre módulos são locais. Os adaptadores de persistência e arquivos ficam atrás de interfaces. A proposta mantém o banco relacional como limite transacional das operações financeiras e não introduz broker de mensagens nem serviços independentes por funcionalidade.

Consequências

Benefícios: menor quantidade de unidades de implantação; transações locais para reserva e pagamento; testes de serviços sem dependência obrigatória do R2; separação de responsabilidades por domínio.

Compromissos: os módulos do backend são publicados e escalados em conjunto; uma falha do processo afeta toda a API; dependências entre pacotes precisam de controle para evitar acoplamento. A indisponibilidade da VPS interrompe o atendimento, mesmo quando os objetos permanecem disponíveis no R2.

Mitigações: contratos de interface explícitos, dependências unidirecionais, testes de integração, logs operacionais e recuperação de banco e arquivos. Backup permite recuperação de dados, mas não representa alta disponibilidade.

Relacionamentos: Figuras 1, 2 e 3; atributos de manutenção, rastreabilidade e desempenho da especificação 2.1.

7. ADR-002 e ADR-003 - Dados e arquivos

ADR-002 - Persistência relacional no MariaDB

Status: Aceito · **Registro:** 09/10/2026

Contexto: pagamentos devem permanecer ligados a contratos, produtores e projetos. Há necessidade de chaves estrangeiras, unicidade de operações e transações. MariaDB 13.0.2 é a tecnologia de banco definida para o sistema.

Alternativas: banco relacional mantém integridade e consultas consolidadas; documentos NoSQL exigiriam controles adicionais para esses vínculos; arquivos e planilhas não oferecem o controle transacional necessário. Essas opções são comparações de adequação ao domínio.

Decisão: utilizar MariaDB com Spring Data JPA/Hibernate. Serviços controlam transações; aprovação e reserva são atômicas. Cada pagamento referencia o compromisso anual, que referencia o contrato, de onde derivam produtor e projeto. Valores são representados por BigDecimal no Java e DECIMAL no banco.

Consequências: integridade referencial e rastreabilidade centralizadas; esquema e migrações precisam de versionamento. A aprovação bloqueia o projeto e cria uma reserva única por contrato e exercício. O pagamento bloqueia o compromisso; transações mantêm a mesma ordem de bloqueios quando combinam recursos. Uma chave de operação única, vinculada ao conteúdo e ao usuário, impede duplicar o mesmo envio. Várias parcelas continuam permitidas.

ADR-003 - Objetos privados no Cloudflare R2

Status: Aceito · **Registro:** 09/10/2026

Contexto: contratos gerados e croquis precisam de armazenamento persistente, vínculo com registros e acesso restrito. O destino definido para os arquivos é o Cloudflare R2.

Alternativas: volume local vincula os objetos ao host e exige recuperação conjunta do servidor; BLOB no banco amplia o volume do banco e de seus backups; armazenamento de objetos separa os arquivos dos dados relacionais.

Decisão: armazenar o conteúdo em bucket privado do R2. O MariaDB guarda arquivo_chave, metadados aplicáveis e o vínculo ao registro; o bucket é configurado no backend. A API autoriza upload e download e utiliza IArquivos. Credenciais e URLs temporárias não são referências persistentes no banco.

Consequências: os objetos não dependem do volume local da API, mas uploads e downloads dependem da conexão com R2. Banco e R2 não compartilham transação: somente confirmar a referência após envio bem-sucedido. Falha ao gravar no banco exige remoção compensatória do objeto novo ou registro para limpeza. Arquivos de contratos aprovados não são sobrescritos.

Recuperação: manter cópia do banco e cópia independente dos objetos, preservando suas chaves. Restaurar ambos e verificar os downloads. O volume persistente do MariaDB permanece necessário.

Relacionamentos: IRepositorios e IArquivos (Figura 1); MariaDB e R2 (Figura 2).

8. Atributos de qualidade

As táticas abaixo atendem aos atributos descritos na seção 6 da especificação 2.1. As verificações indicam evidências esperadas, sem pressupor resultados de testes já executados.

Atributo	Tática arquitetural	Verificação
Desempenho	Cálculo local na API; leitura dos parâmetros necessários; índices nas chaves e filtros; consultas paginadas.	Medir a pontuação e o valor apresentados em até cinco segundos, com ambiente e volume de dados registrados.
Segurança	Autenticação e RBAC no backend; autorização por registro e arquivo. Contas sem cadastro público; administrador inicial provisionado.	Tentar operações com perfil indevido, conta inativa e identificador de outro produtor; negar acesso.
Usabilidade	Angular responsivo; erros por campo; salvamento progressivo do diagnóstico; retorno claro sobre falha de conexão.	Executar o fluxo em computador e celular, retomando o rascunho persistido. Não presumir funcionamento offline.
Rastreabilidade	Parâmetros usados preservados no cálculo; vínculos por FK; autoria e data dos registros; pagamentos ligados à origem.	Alterar parâmetros e comprovar que cálculos e contratos anteriores permanecem inalterados.
Privacidade	Dados pessoais restritos aos perfis autorizados; R2 privado; DTOs sem senha/hash; logs sem credenciais ou documentos completos.	Verificar respostas, logs e downloads; assegurar que produtor consulta exclusivamente os próprios dados.
Manutenção	Fórmula única, configuração por projeto, pacotes coesos, interfaces e scripts versionados de alteração do banco.	Modificar parâmetros sem alterar algoritmo; testar serviços com substitutos dos adaptadores.

Quadro 3 - Atributos de qualidade, táticas e evidências de verificação.

8.1 Autenticação e integridade financeira

A autenticação utiliza sessão no backend, cookie HttpOnly/Secure e proteção CSRF. O navegador acessa /api/v1 pela mesma origem do frontend, encaminhada pelo proxy à API (seção 12). A conta de produtor possui vínculo com o cadastro correspondente. A sessão em memória é compatível com a instância única; reinício exige novo login. O administrador não herda automaticamente operações de gestor e técnico.

A consulta financeira separa recursos, caixa, comprometimento pendente e disponível. Caixa = recursos − pagamentos; pendência = compromissos − pagamentos; disponível = caixa − pendência. O pagamento parcial reduz caixa e pendência no mesmo valor. Sem movimentos, os totais são zero; uma inconsistência deve indicar consolidação incompleta (UC08).

A fórmula é preservada: **Valor anual = (Valor da terra por hectare × Percentual) × Pontuação × Área contratada**. A API recupera os parâmetros do projeto e conserva a memória do resultado.

9. Rastreabilidade entre requisitos e arquitetura

Requisito / cenário	Responsabilidade arquitetural
RF01 / UC01	cadastros: produtor e propriedade; persistência relacional.
RF02 / UC02	cadastros: fontes e ausência explícita de fontes hídricas.
RF03 / UC03	diagnostico: respostas progressivas e retomada.
RF04 / UC04	diagnostico: apuração e correção controlada.
RF05 / UC05	calculo: fórmula, validação de área e memória do resultado.
RF06 / US01	configuracao: parâmetros completos por projeto, geridos pelo administrador.
RF07 / US02	cadastros / arquivos: croqui vinculado à propriedade e à área contratada.
RF08 / UC06	contratos / arquivos: gerar instrumento e referenciar objeto R2.
RF09 / US03	contratos: registro separado da assinatura simulada.
RF10 / UC07	financeiro: parcelas, idempotência e vínculo ao contrato/projeto/produtor.
RF11 / UC08	financeiro: totais separados e indicação de inconsistências.
RF12 / US04	consultas: relatório por período, projeto e sub-bacia.
RF13 / US05	consultas: indicadores consolidados.
RF14 / US06	contratos / consultas: vigência e sinalização de vencimento.
RF15 / UC09	acesso: autenticação e autorização por perfil.
RF16 / US07	consultas / acesso: histórico exclusivo do produtor autenticado.
RF17 / US08	cadastros: visitas técnicas de acompanhamento.
RF18 / UC10	financeiro: aporte ou saldo inicial manual, com origem e projeto.

Quadro 4 - Rastreabilidade dos 18 requisitos funcionais.

9.1 Coerência entre as visões

FrontendAngular corresponde ao build servido pelo Nginx e executado no navegador. ApiPSA e os adaptadores de segurança, persistência, arquivos e cache compõem psa-gestor-api.jar. MariaDB mantém os registros e referências; R2 mantém os objetos. Os módulos da seção 3 são organizados pelos pacotes da Figura 3.

Técnico e gestor podem calcular PSA, conforme UC05. O administrador configura parâmetros (US01); técnico e gestor confirmam ou informam os parâmetros permitidos pelo perfil no UC05. O croqui permanece vinculado à propriedade e ao contrato; o R2 armazena seu conteúdo e não substitui esses relacionamentos de negócio.

10. ADR-004 - Cache seletivo com Redis

Cache descartável com expiração automática

Status: Adotado nesta arquitetura · **Registro:** 09/10/2026

Contexto: o painel pode repetir agregações sobre produtores e área protegida. O cache evita repetir essas consultas quando há reutilização; o benefício será verificado por medição. Saldos, pagamentos e valores utilizados no cálculo exigem leitura dos registros persistidos.

Alternativas: leitura direta no MariaDB reduz componentes; cache local reduz comunicação de rede, mas compartilha a memória da JVM; Redis permite limitar e observar o cache em processo separado, com custo operacional adicional.

Decisão: manter Redis como cache de indicadores não financeiros, com leitura sob demanda e expiração automática. O **TTL inicial é de 60 segundos** após o armazenamento. Não há registro auxiliar de chaves, bloqueio distribuído nem invalidação coordenada com transações de negócio.

Situação	Comportamento
Antes da consulta	Validar sessão, conta ativa, perfil e acesso ao projeto. A chave inclui versão do formato, usuário, perfil, projeto e filtros normalizados.
Entrada válida	Retornar o resultado com data e hora de apuração. A tela informa quando os indicadores foram atualizados.
Entrada ausente ou expirada	Consultar o MariaDB; tentar guardar o resultado com TTL; responder mesmo quando a escrita no Redis falhar.
Alteração dos dados	A próxima leitura após expiração refaz a agregação. Os indicadores podem ficar temporariamente defasados; a operação de escrita retorna seu próprio resultado persistido.
Redis indisponível	Aplicar **tempo limite inicial de 200 ms** e consultar o MariaDB. O erro de cache não provoca falha na operação de negócio.
Dados excluídos	Saldos, valor médio pago, relatórios financeiros, pagamentos, parâmetros de cálculo, permissões, sessões e documentos não são armazenados no cache.

10.1 Limites e tratamento de falhas

Parâmetros iniciais: maxmemory de **32 MiB**, política **allkeys-lru** e contêiner limitado a **64 MiB**. A diferença atende às estruturas internas e conexões, sujeita a medição. Desabilitar AOF e snapshots, pois todas as entradas são reconstruíveis; não publicar a porta do Redis no host.

Uma falha no Redis aciona leitura direta, sem repetir comandos de cache dentro da mesma requisição. Após reinício, as entradas existentes continuam sujeitas ao TTL. Mudanças no formato dos dados usam novo prefixo de chave. Autorização é sempre verificada no backend, nunca inferida do conteúdo em cache.

Consequências: implementação simples, dados temporários identificados e funcionamento independente da disponibilidade do cache. Há consumo adicional de memória e comunicação de rede; a manutenção do cache depende da comparação entre latência, taxa de acerto e custo. É possível desativá-lo por configuração.

11. Recursos e desempenho

Os valores abaixo são parâmetros iniciais de implantação, não consumo medido nem garantia de capacidade. O ajuste considera a memória efetivamente disponível na VPS e a execução dos fluxos do sistema.

Serviço	Limite inicial	Ajuste principal
Backend Java	**640 MiB**	Heap inicial de 128 MiB e máximo de 384 MiB; reservar o restante para memória nativa, classes e threads.
MariaDB	**448 MiB**	Buffer pool inicial de 192 MiB; até 20 conexões no servidor, com pool da API limitado a 5.
Nginx Proxy Manager	**256 MiB**	SQLite próprio; dados e certificados em volumes persistentes.
Redis	**64 MiB**	Cache de 32 MiB; sem persistência; TTL de 60 segundos.
Nginx estático	**48 MiB**	Arquivos pré-compilados; compressão e cache de recursos com hash.
Sistema e margem	**592 MiB**	Os limites somam **1.456 MiB** dos 2.048 MiB da VPS, restando cerca de 29% para Linux, Docker e picos. A margem é apertada: confirmar a RAM utilizável antes da implantação e medir o consumo real.

11.1 Redução de trabalho por requisição

Frontend: compilação de produção, carregamento de rotas sob demanda, imagens dimensionadas e recursos com hash e cache prolongado. O index.html deve ser revalidado; respostas autenticadas não recebem cache público. Compilar fora da VPS e publicar apenas o artefato final.

API: DTOs enxutos, paginação com **máximo inicial de 100 registros**, validação antes de acessar o banco e tempo limite para chamadas externas. Iniciar o servidor HTTP com **até 20 threads** e fila limitada; medir antes de ampliar. Geração de PDF e upload possuem limites de tamanho e execução para evitar retenção excessiva de memória.

Banco: consultas parametrizadas, projeções para listagens e carregamento explícito das relações necessárias. Desabilitar Open Session in View. Criar índices alinhados aos filtros de projeto, produtor, contrato, situação e data; verificar planos com EXPLAIN e evitar consultas N+1. Migrações são versionadas.

11.2 Evidências de desempenho

Registrar latências mediana e p95, memória dos contêineres, pausas de coleta de lixo, espera pelo pool, consultas lentas e acertos no Redis. Comparar consultas com cache frio, aquecido e indisponível. Validar o cálculo e a apresentação do resultado **em até cinco segundos**, conforme o requisito existente.

Executar os testes com base representativa e registrar volume, ambiente e resultados. Limites excessivamente baixos podem encerrar processos; a configuração só é considerada validada após testes de geração de documentos, backup e recuperação, além das consultas.

12. Segurança e publicação web

Endereço / caminho	Encaminhamento no Nginx Proxy Manager
psagestor.duckdns.org	HTTPS: interface Angular servida pelo Nginx estático.
psagestor.duckdns.org/api/v1/	HTTPS: encaminhamento à API, preservando o caminho /api/v1/. Esta é a origem utilizada pelo navegador.
api-psagestor.duckdns.org	HTTPS: acesso direto à mesma API para clientes autorizados e validação técnica. Não concede acesso anônimo aos dados.

12.1 Sessão e controle de acesso

O Nginx Proxy Manager centraliza TLS e roteamento; o Nginx estático entrega apenas o artefato Angular, permitindo atualizá-lo sem alterar o proxy. Os dois nomes DuckDNS são origens distintas. O frontend utiliza chamadas relativas em /api/v1 para manter a sessão na mesma origem e evitar dependência de cookies entre sites. O domínio próprio da API permanece publicado; o caminho adicional não duplica serviços nem regras.

O cookie de sessão é restrito ao host, com **Secure, HttpOnly, SameSite=Lax** e caminho /. O token CSRF é fornecido ao Angular e enviado em cabeçalho nas operações de escrita; ele é distinto do cookie de sessão. O backend valida o token e renova a sessão no login. Logout invalida a sessão. Prazo inicial de inatividade: **30 minutos**.

Contas são criadas pelo administrador; não existe cadastro público. Senhas recebem hash adaptativo com salt; credenciais iniciais são substituídas. A API verifica perfil, situação da conta e vínculo ao recurso em todas as operações, inclusive downloads. Alterações de perfil ou desativação invalidam as sessões correspondentes.

12.2 Superfície de exposição

Publicar apenas **HTTP/80** para redirecionamento e emissão de certificado, e **HTTPS/443** para aplicação. Restringir SSH por chave e acesso administrativo. A interface do Nginx Proxy Manager fica vinculada ao endereço local, acessível por túnel SSH; MariaDB, Redis e a porta interna da API não são publicados no host.

Docker Compose define redes separadas para entrada, aplicação e dados. O proxy acessa os servidores web; somente a API acessa banco e cache. A API mantém saída HTTPS para o R2. Cabeçalhos de encaminhamento são aceitos apenas do proxy confiável. CORS não utiliza origem curinga com credenciais.

12.3 Arquivos, erros e segredos

Aplicar limite inicial de **10 MiB por arquivo**, com ajuste correspondente no proxy e na API. Validar extensão, tipo e assinatura do conteúdo conforme a lista permitida; gerar chaves internas sem utilizar caminhos enviados pelo usuário. Downloads exigem autorização, usam transmissão em fluxo e cabeçalhos seguros.

Segredos ficam fora do repositório e das imagens, em arquivos com permissões restritas. Erros retornam código, mensagem e identificador de requisição, sem stack trace ou credenciais. Limitar tentativas de login e registrar eventos de acesso, sem bloqueios permanentes automáticos.

13. Operação e recuperação

13.1 Publicação reproduzível

Dockerfiles, configuração de implantação e migrações de banco são artefatos versionados previstos para o repositório. A validação executa testes, compila backend e frontend e gera imagens identificadas pela revisão do código. A VPS recebe imagens prontas; compiladores e ferramentas de desenvolvimento não precisam permanecer em execução.

Fixar versões das imagens e dependências no manifesto de implantação. **Java 25, Spring Boot 4.1.1, Angular 21 e MariaDB 13.0.2** compõem a base definida. Para Redis, Nginx e Nginx Proxy Manager, registrar a versão exata homologada e seu digest antes da publicação; evitar tags flutuantes.

Executar backup antes de migrações, aplicar alterações compatíveis e verificar saúde da API e fluxo de login. Reverter a imagem somente quando o esquema continuar compatível; caso contrário, utilizar correção progressiva ou restauração planejada. A instância única pode exigir uma breve interrupção durante a atualização.

13.2 Monitoramento proporcional ao ambiente

A arquitetura utiliza Spring Boot Actuator e métricas da JVM para fornecer indicadores internos de saúde, memória e requisições. Restringir endpoints de gerenciamento. Uma falha do Redis sinaliza degradação, mas não retira a API de serviço; indisponibilidade do MariaDB impede operações que dependem de persistência.

Logs incluem data, nível e identificador de requisição; eventos de negócio registram usuário, operação e recurso no MariaDB. Configurar rotação inicial de **três arquivos de 10 MiB** por contêiner e acompanhar espaço livre, certificados e falhas de backup. A política de reinício não substitui diagnóstico de falhas repetidas.

13.3 Cópias e restauração

Objeto	Política inicial proposta
MariaDB	Cópia lógica **diária** consistente para tabelas InnoDB, criptografada e enviada para destino externo; manter **sete cópias diárias**. Não executar migração durante a cópia.
R2 e documentos	Preservar os objetos referenciados e manter cópia independente. Um bucket de backup separado do bucket operacional reduz exclusões acidentais, mas não elimina risco comum do provedor.
Proxy e implantação	Copiar configuração, certificados e manifesto; proteger credenciais. Redis não exige backup, pois não mantém dados de negócio.
Restauração	Interromper escritas; restaurar banco e objetos compatíveis; conferir chaves, downloads e saldos; limpar o cache; executar teste de login e registro controlado.

A política diária admite perda dos registros posteriores à última cópia válida. O tempo de recuperação será medido em ensaio. Manter folga no disco de 70 GB, remover imagens antigas com critério e limitar arquivos temporários. A VPS única é um ponto de falha; backups não oferecem alta disponibilidade.

14. Sequência de consulta com cache

O diagrama apresenta uma consulta autorizada a indicadores não financeiros. Requisições sem sessão ou sem permissão são rejeitadas antes do acesso ao Redis ou ao MariaDB.

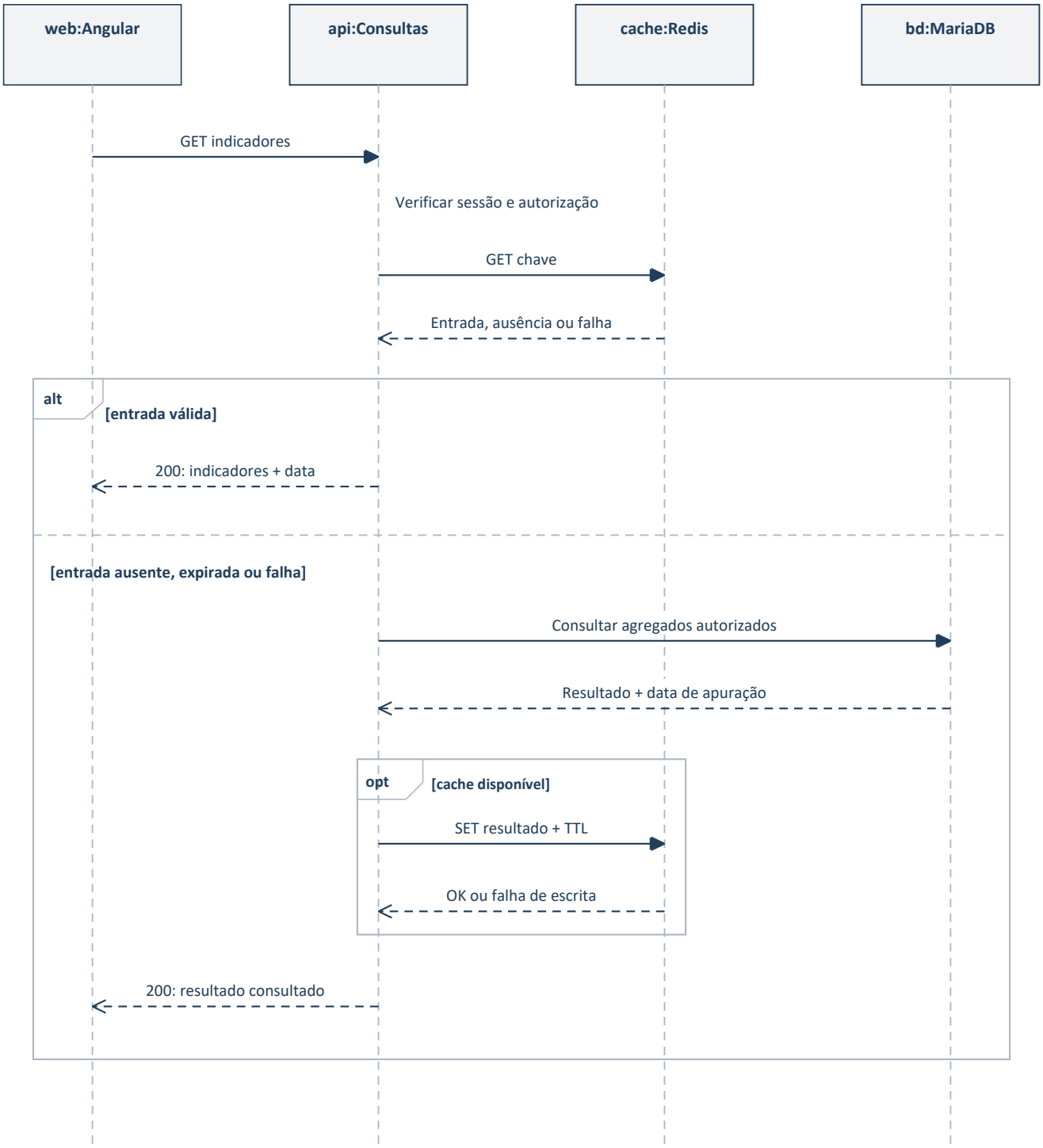


Figura 4 - Leitura com alternativas de acerto e consulta direta.

Setas contínuas representam chamadas; setas tracejadas representam retornos ou resultados de tentativa. Falha de conexão é detectada pelo cliente da API. O fragmento alt escolhe um caminho; opt executa a gravação apenas quando aplicável. A escrita no Redis não é requisito para responder.

15. Sequência de pagamento

O fluxo considera gestor autenticado, contrato e projeto identificados. A chave de operação é única e vinculada ao usuário e ao conteúdo. Registra-se o pagamento informado, sem transferência bancária.

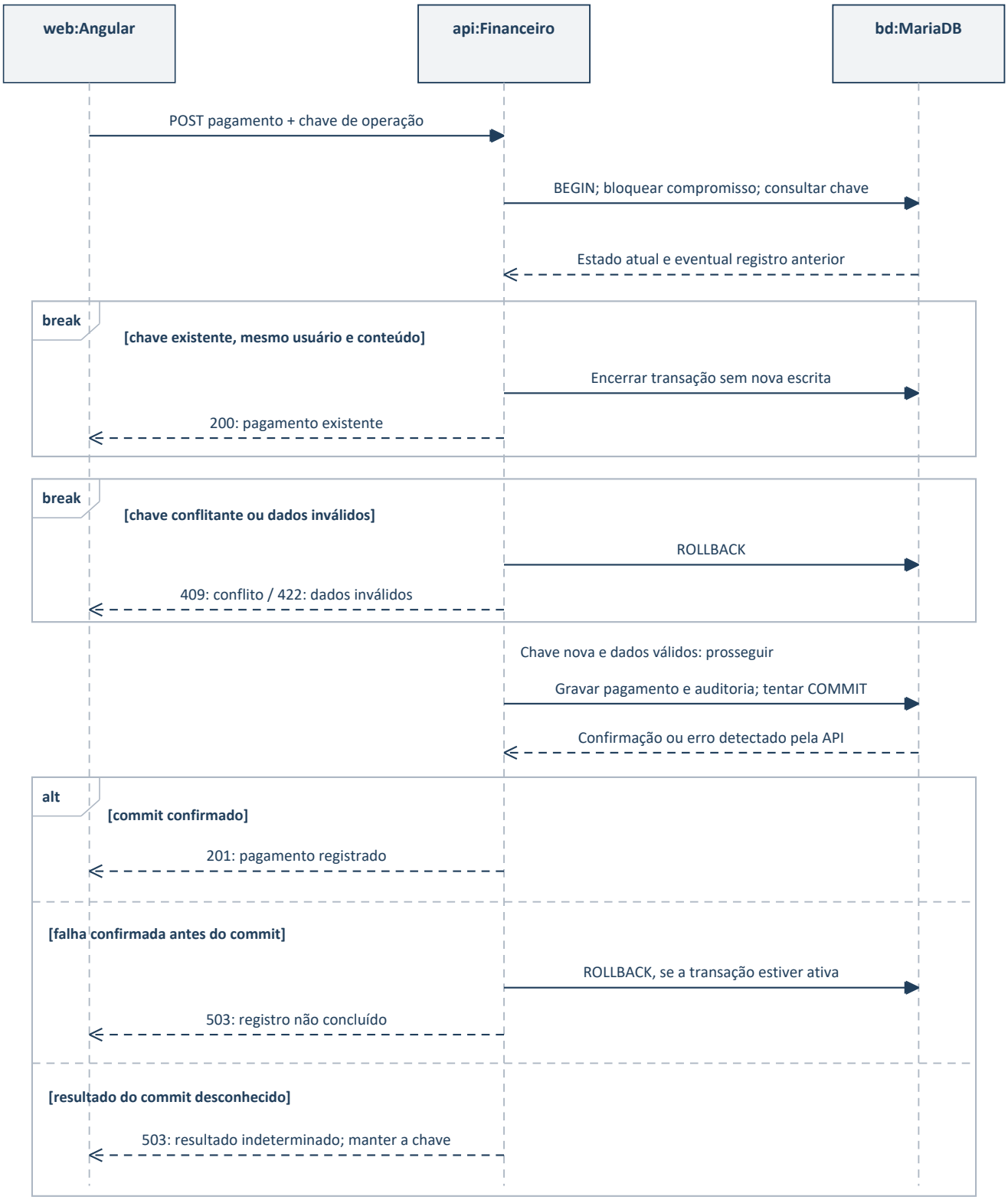


Figura 5 - Registro de pagamento, repetição e resultados da transação.

O fragmento break encerra o fluxo quando sua condição é verdadeira. A restrição única protege envios concorrentes; o bloqueio dura até o fim da transação. Uma falha de conexão não confirma rollback. Em resultado indeterminado, consultar ou repetir com a mesma chave após recuperar a conexão, sem duplicar o pagamento.

16. Decisões consolidadas

Tema	Definição arquitetural e vínculo funcional
Cálculo e configuração	Fórmula única; memória dos parâmetros efetivamente utilizados (RN03 a RN05). Administrador configura; técnico e gestor executam o cálculo dentro das permissões do UC05.
Contratos e correções	Gestor gera o instrumento, registra a vigência e pode retornar aos dados de origem para corrigir e gerar novamente (UC06/A1). A assinatura simulada permanece separada (US03).
Comprometimento	A formalização e aprovação reservam o valor anual. Aprovação e criação do compromisso são atômicas e não podem duplicar a reserva do mesmo contrato e exercício.
Pagamentos e saldo	Parcelas são permitidas (UC07/A1). Pagamento referencia contrato, produtor e projeto; relatórios separam recursos, comprometimento e valores pagos (RN10 e RN11).
Vencimento	Registrar a vigência e apresentar contratos próximos do vencimento (RF14/US06). O aviso permite organizar renovações; não implica renovação automática.
Infraestrutura	VPS Loophost com 2 vCPUs, 2 GB e 70 GB; Docker Compose; Nginx Proxy Manager; MariaDB; Redis seletivo; R2 privado. Domínios e encaminhamentos definidos na seção 12.